

Cigarettes and Whisky

$$(P \wedge Q) s = (P s) \text{ and } (Q s)$$

$$(P \vee Q) s = (P s) \text{ or } (Q s)$$

$$(\neg P) s = \text{not } (P s)$$

$$\img alt="cigarette pack" data-bbox="187 523 245 584"/> s = s \geq \pounds 5$$

$$\img alt="whisky bottle" data-bbox="212 583 245 652"/> s = s \geq \pounds 20$$

$$\begin{aligned} (\img alt="cigarette pack" data-bbox="100 700 155 760"/> \wedge \img alt="whisky bottle" data-bbox="175 700 210 760"/>) s &= (\img alt="cigarette pack" data-bbox="315 700 370 760"/> s) \text{ and } (\img alt="whisky bottle" data-bbox="480 700 515 760"/> s) \\ &= s \geq \pounds 5 \text{ and } s \geq \pounds 20 \\ &= s \geq \pounds 20 \end{aligned}$$

Hardware & Software Verification

John Wickerson

Lecture 8: Hoare Logic and Separation Logic
25 November 2025

Today's lecture

- Some underlying principles of software verification.
- A brief history of the field from around 1969 (Hoare Logic) to around 2001 (Separation Logic).

Hoare Logic

- Invented by Tony Hoare in 1969.
- A mathematical system based on annotating program code with **assertions** that must hold whenever execution reaches that point.
- The basic unit is the Hoare triple, written $\{ p \} C \{ q \}$.



C.A.R. (Tony) Hoare
(1934–)

Hoare triples

- What does $\{ p \} C \{ q \}$ mean?
 - It's more-or-less equivalent to the following Dafny code:

```
assume p;  
C;  
assert q;
```
 - If C begins execution in a state satisfying p , then any final state it reaches will satisfy q .
 - C can be an entire program, or just a single instruction.

Rules of Hoare logic

$$\frac{\{p\} C_1 \{q\} \quad \{q\} C_2 \{r\}}{\{p\} C_1 ; C_2 \{r\}} \text{SEQ}$$

$$\frac{\{p \wedge b\} C_1 \{q\} \quad \{p \wedge \neg b\} C_2 \{q\}}{\{p\} \text{if } b \text{ then } C_1 \text{ else } C_2 \{q\}} \text{IF}$$

$$\frac{p \Rightarrow q[e/x]}{\{p\} x := e \{q\}} \text{ASS}$$

$$\frac{p \Rightarrow I \quad \{I \wedge b\} C \{I\} \quad I \wedge \neg b \Rightarrow q}{\{p\} \text{while } b \text{ do } C \{q\}} \text{WHILE}$$

$$\begin{array}{c}
\frac{}{\{ \text{true} \}} \text{ASS} \quad \frac{}{\{ x=0 \}} \text{ASS} \quad \frac{}{\{ 2x=y \wedge x \leq 10 \wedge x < 10 \}} \text{ASS} \quad \frac{}{\{ 2(x-1)=y \wedge x-1 < 10 \}} \text{ASS} \\
x := 0 \quad y := 0 \quad x := x+1 \quad y := y+2 \\
\frac{}{\{ x=0 \}} \text{ASS} \quad \frac{}{\{ x=0 \wedge y=0 \}} \text{ASS} \quad \frac{}{\{ 2(x-1)=y \wedge x-1 < 10 \}} \text{ASS} \quad \frac{}{\{ 2x=y \wedge x \leq 10 \}} \text{SEQ} \\
\frac{}{\{ \text{true} \}} \text{SEQ} \quad \frac{}{\{ 2x=y \wedge x \leq 10 \wedge x < 10 \}} \text{SEQ} \quad \frac{}{\{ 2x=y \wedge x \leq 10 \}} \text{WHILE} \\
x := 0; y := 0 \quad x := x+1; y := y+2 \quad \text{while } x < 10 \text{ do } (x := x+1; y := y+2) \\
\frac{}{\{ x=0 \wedge y=0 \}} \text{SEQ} \quad \frac{}{\{ x=10 \wedge y=20 \}} \text{SEQ} \\
\frac{}{\{ \text{true} \}} \text{SEQ}
\end{array}$$

$x := 0; y := 0;$
 $\text{while } x < 10 \text{ do } (x := x+1; y := y+2)$
 $\{ x=10 \wedge y=20 \}$

Verification conditions:

$$\text{true} \Rightarrow 0=0$$

$$x=0 \Rightarrow x=0 \wedge 0=0$$

$$x=0 \wedge y=0 \Rightarrow 2x=y \wedge x \leq 10$$

$$2x=y \wedge x \leq 10 \wedge \neg(x < 10) \Rightarrow x=10 \wedge y=20$$

$$2x=y \wedge x \leq 10 \wedge x < 10 \Rightarrow 2((x+1)-1)=y \wedge (x+1)-1 < 10$$

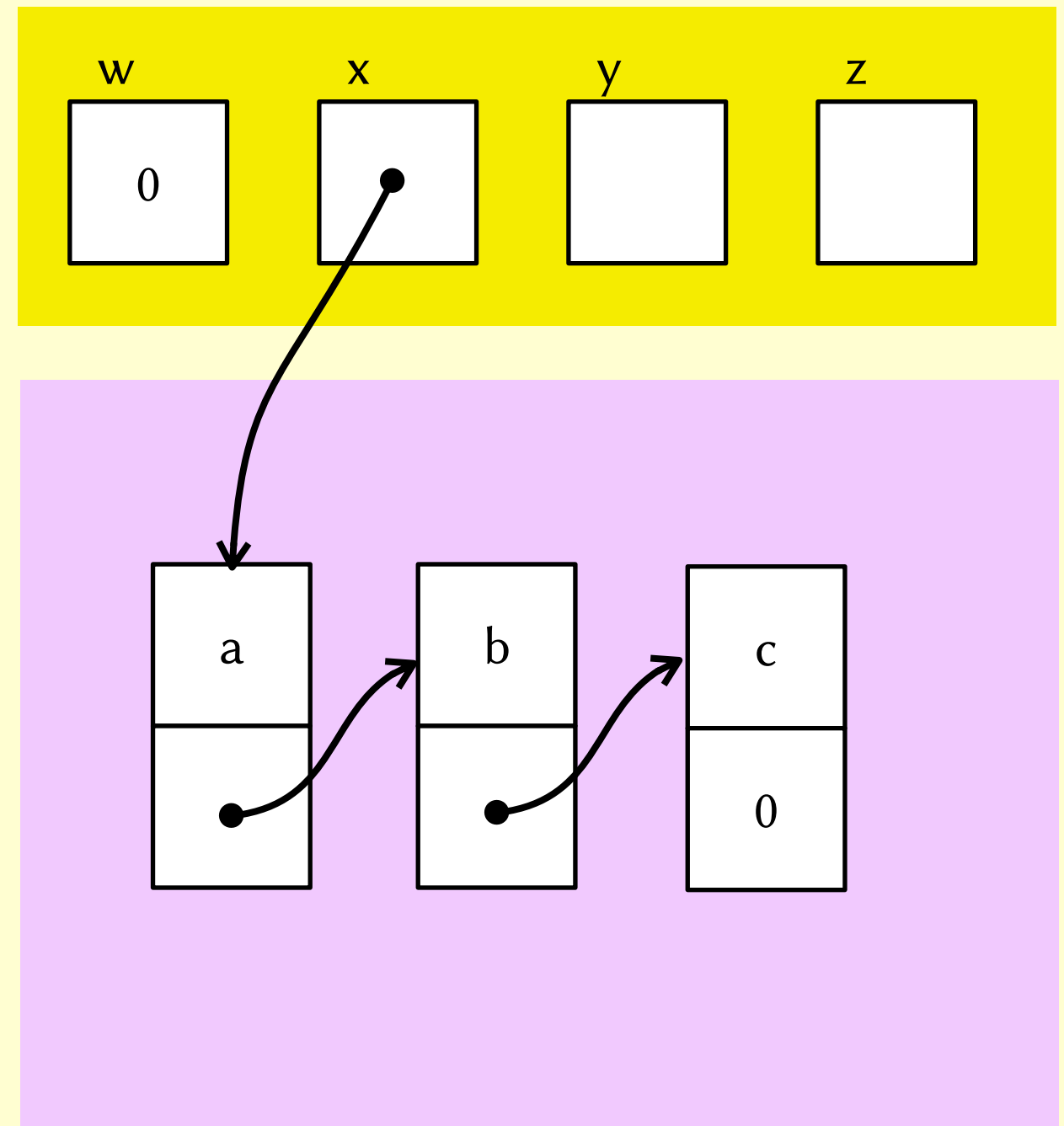
$$2(x-1)=y \wedge x-1 < 10 \Rightarrow 2x=(y+2) \wedge x \leq 10$$

A challenge for Hoare

- Hoare logic struggles to reason about heap-allocated data structures like linked lists.

List reverse

→ $\{list\ \delta\ x\}$
w := 0;
while (x ≠ 0) do {
 z := [x+1];
 [x+1] := w;
 w := x;
 x := z;
}
 $\{list\ -\delta\ w\}$



List reverse

$\{list\ \delta\ x\}$

$w := 0;$

while ($x \neq 0$) do {

$z := [x+1];$

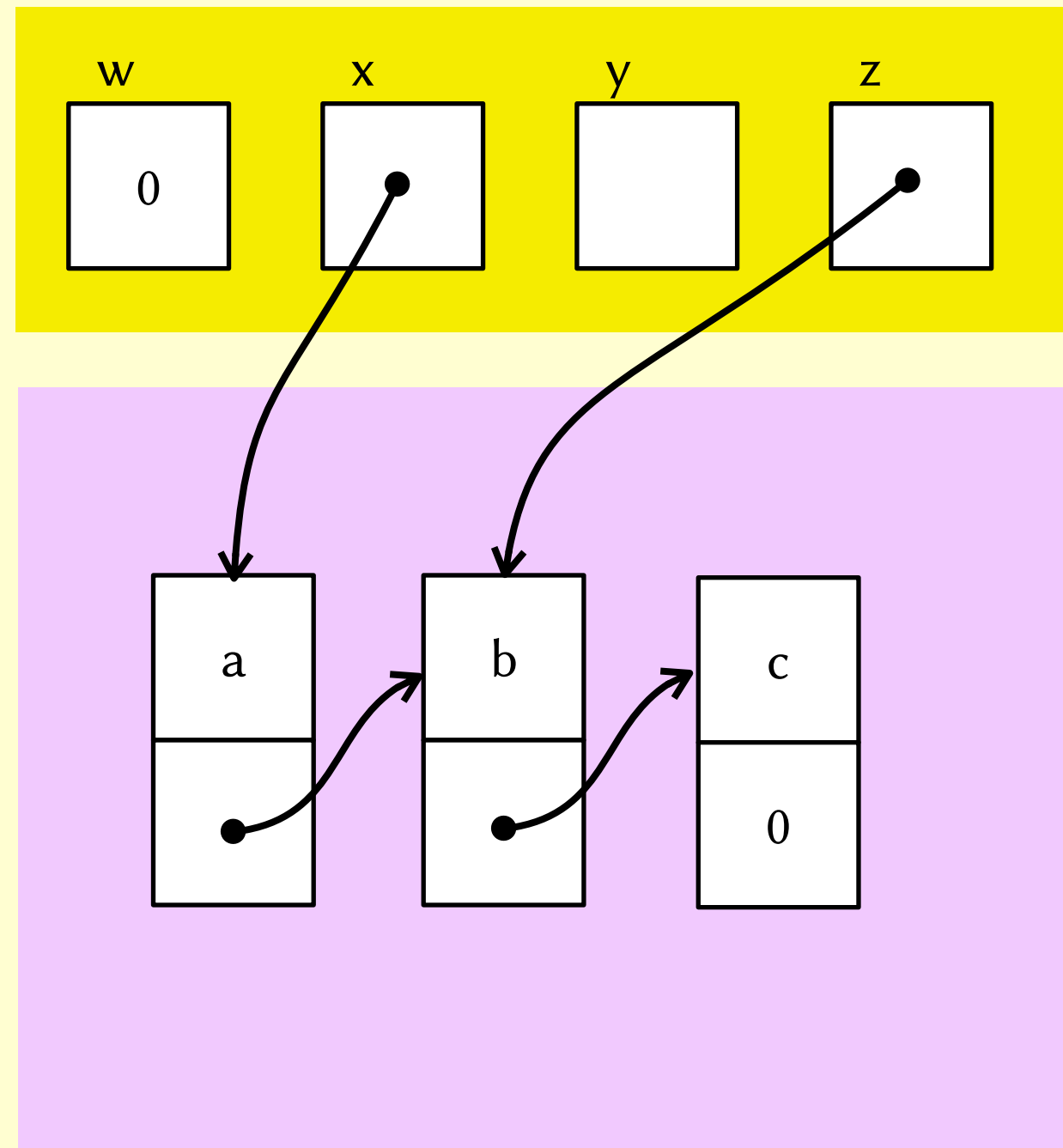
$[x+1] := w;$

$w := x;$

$x := z;$

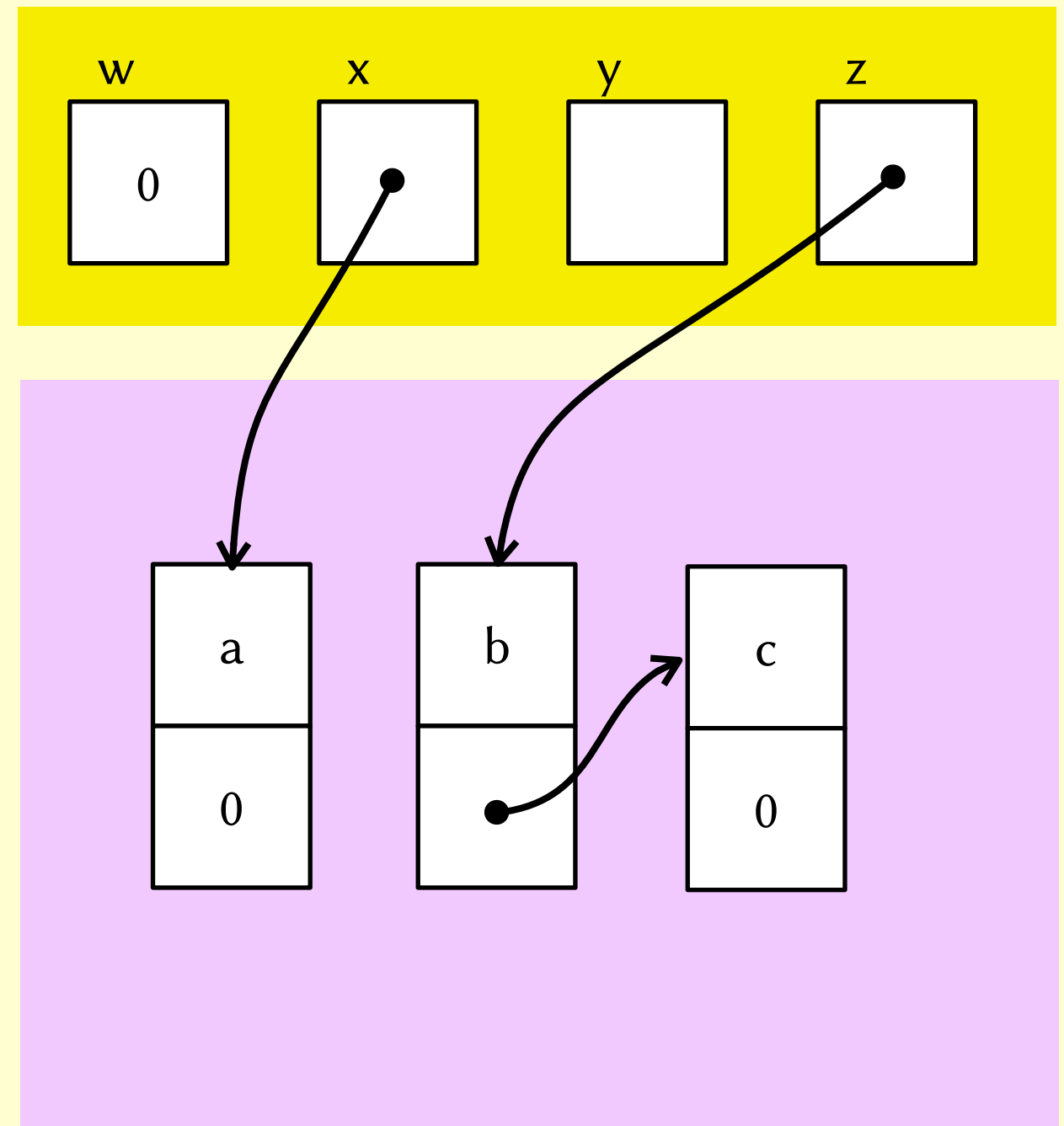
}

$\{list\ -\delta\ w\}$



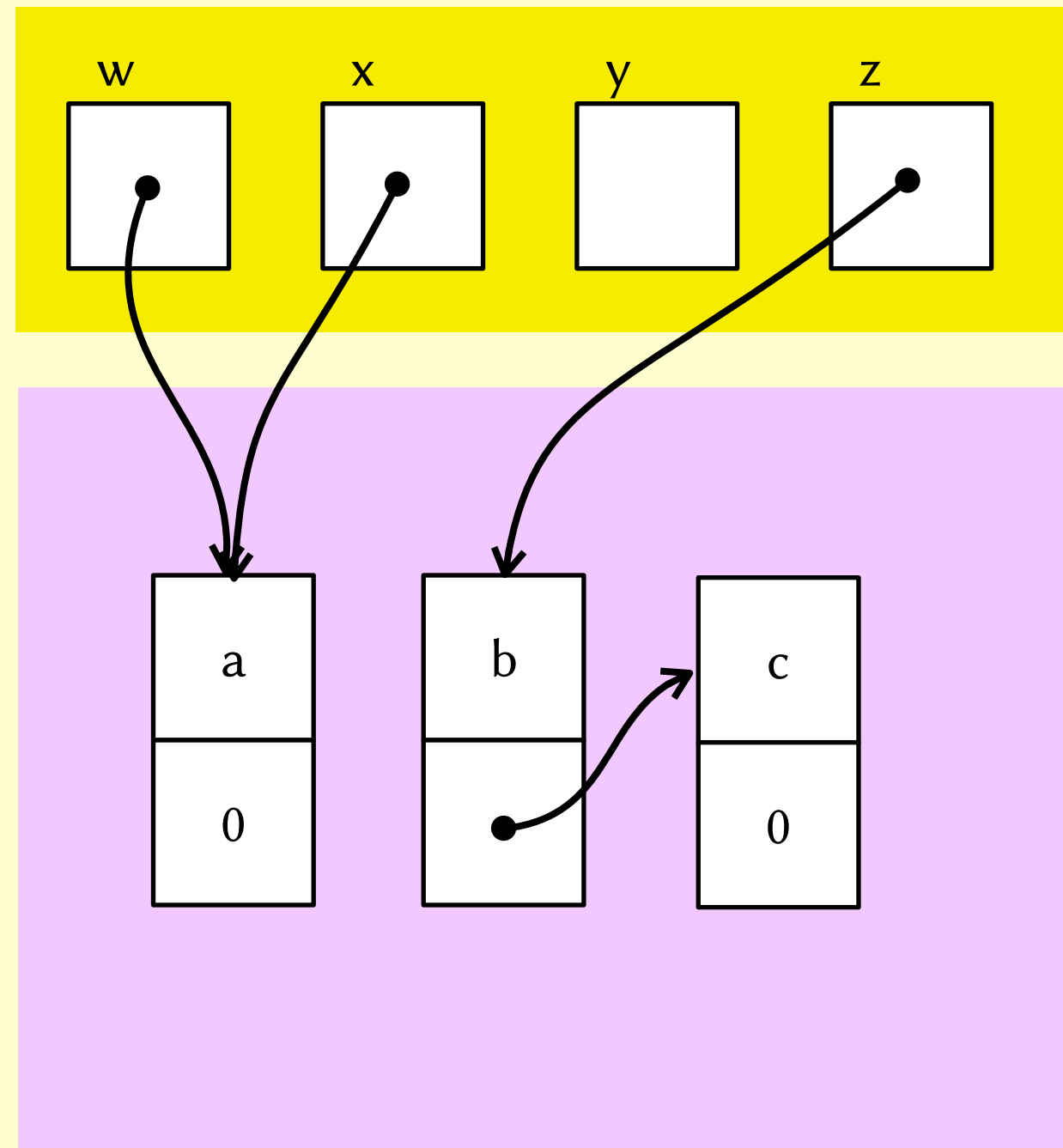
List reverse

$\{list\ \delta\ x\}$
 $w := 0;$
while $(x \neq 0)$ do {
 $z := [x+1];$
 $[x+1] := w;$
 $w := x;$
 $x := z;$
}
 $\{list\ -\delta\ w\}$



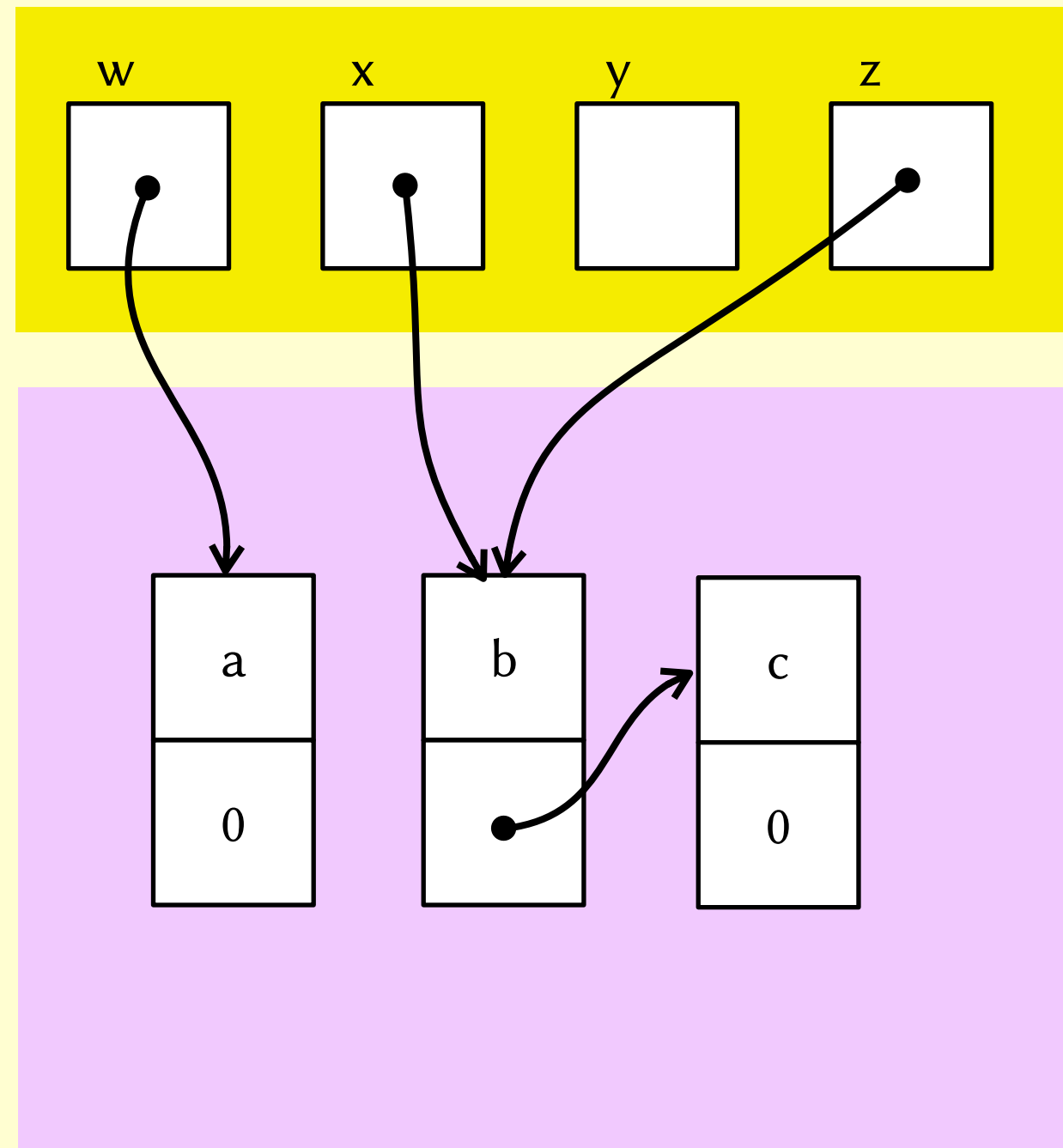
List reverse

$\{list\ \delta\ x\}$
 $w := 0;$
while $(x \neq 0)$ do {
 $z := [x+1];$
 $[x+1] := w;$
 $w := x;$
 $x := z;$
}
 $\{list\ -\delta\ w\}$



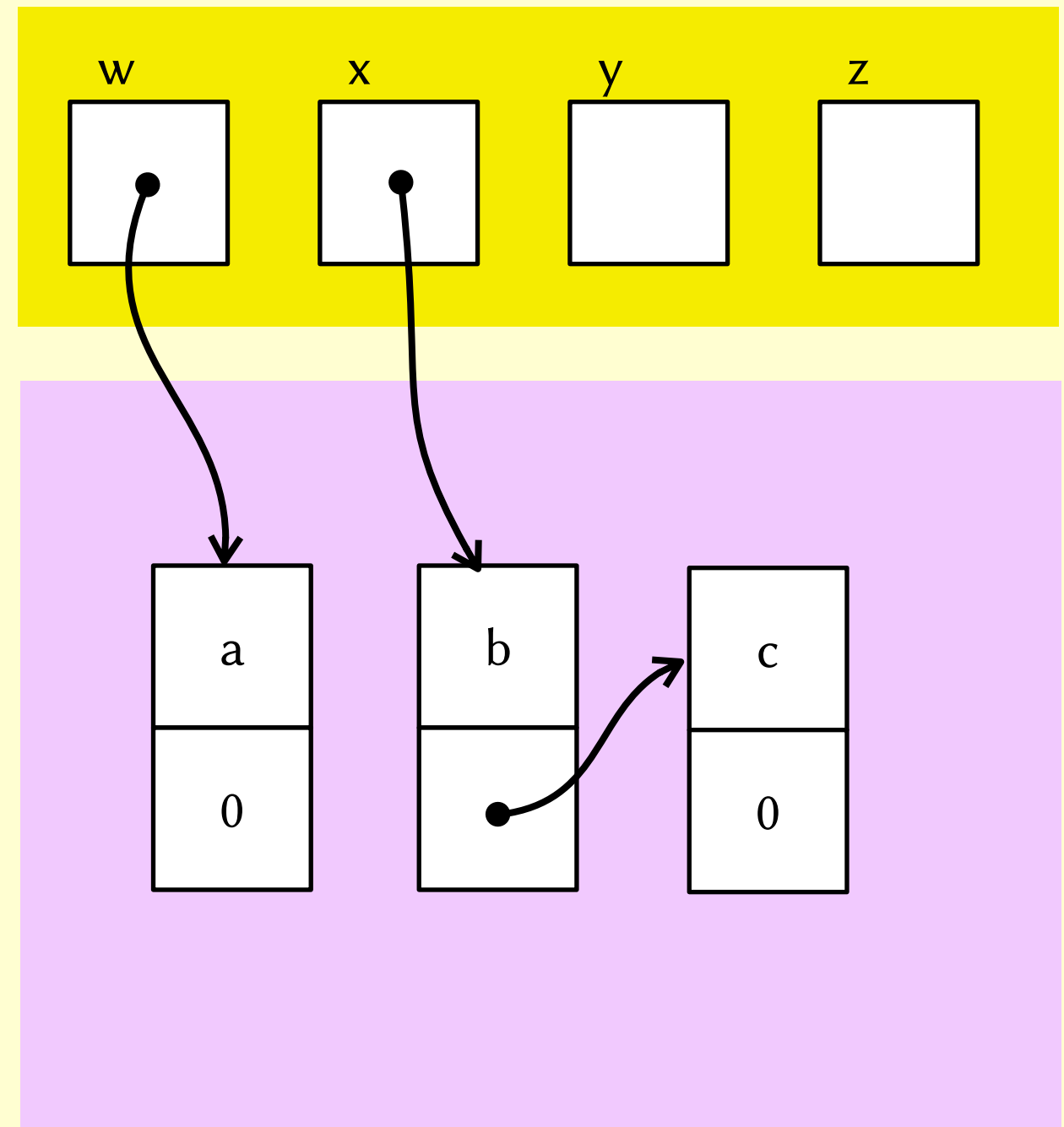
List reverse

$\{list\ \delta\ x\}$
 $w := 0;$
while $(x \neq 0)$ do {
 $z := [x+1];$
 $[x+1] := w;$
 $w := x;$
 $x := z;$
}
 $\{list\ -\delta\ w\}$



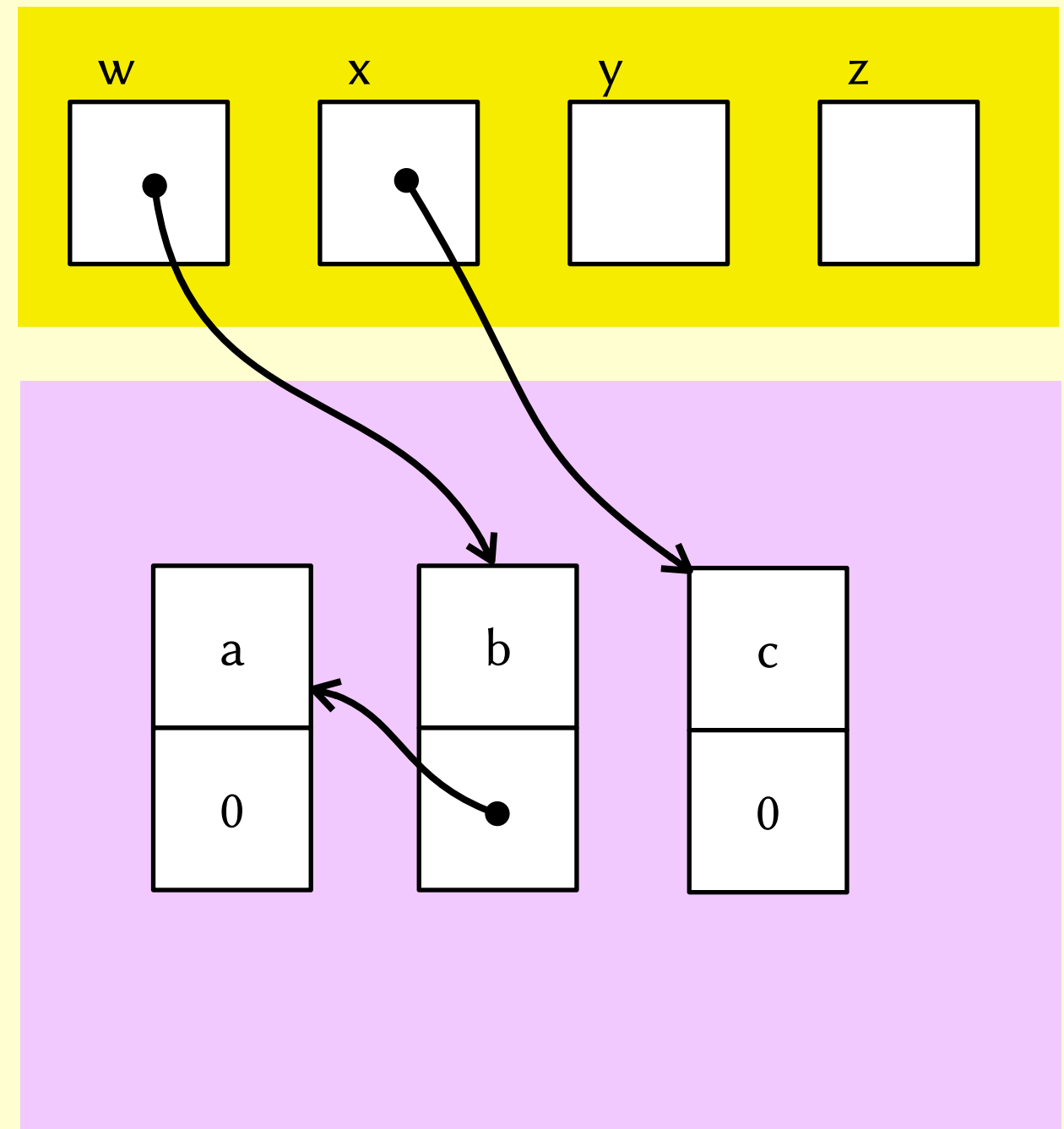
List reverse

```
{list  $\delta$  x}  
w := 0;  
while (x  $\neq$  0) do {  
  z := [x+1];  
  [x+1] := w;  
  w := x;  
  x := z;  
}  
{list  $-\delta$  w}
```



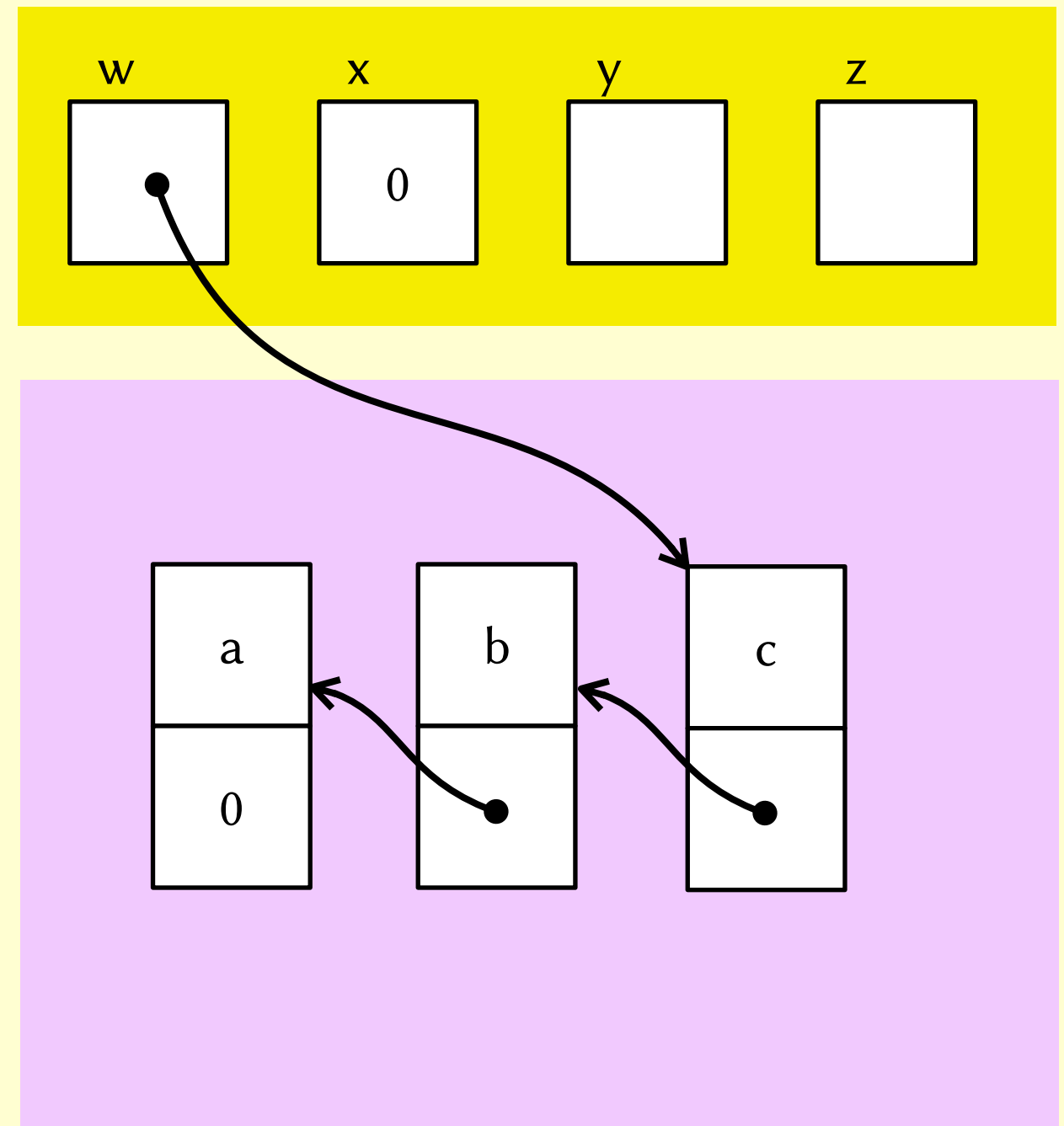
List reverse

```
{list  $\delta$  x}  
w := 0;  
while (x  $\neq$  0) do {  
  z := [x+1];  
  [x+1] := w;  
  w := x;  
  x := z;  
}  
{list  $-\delta$  w}
```



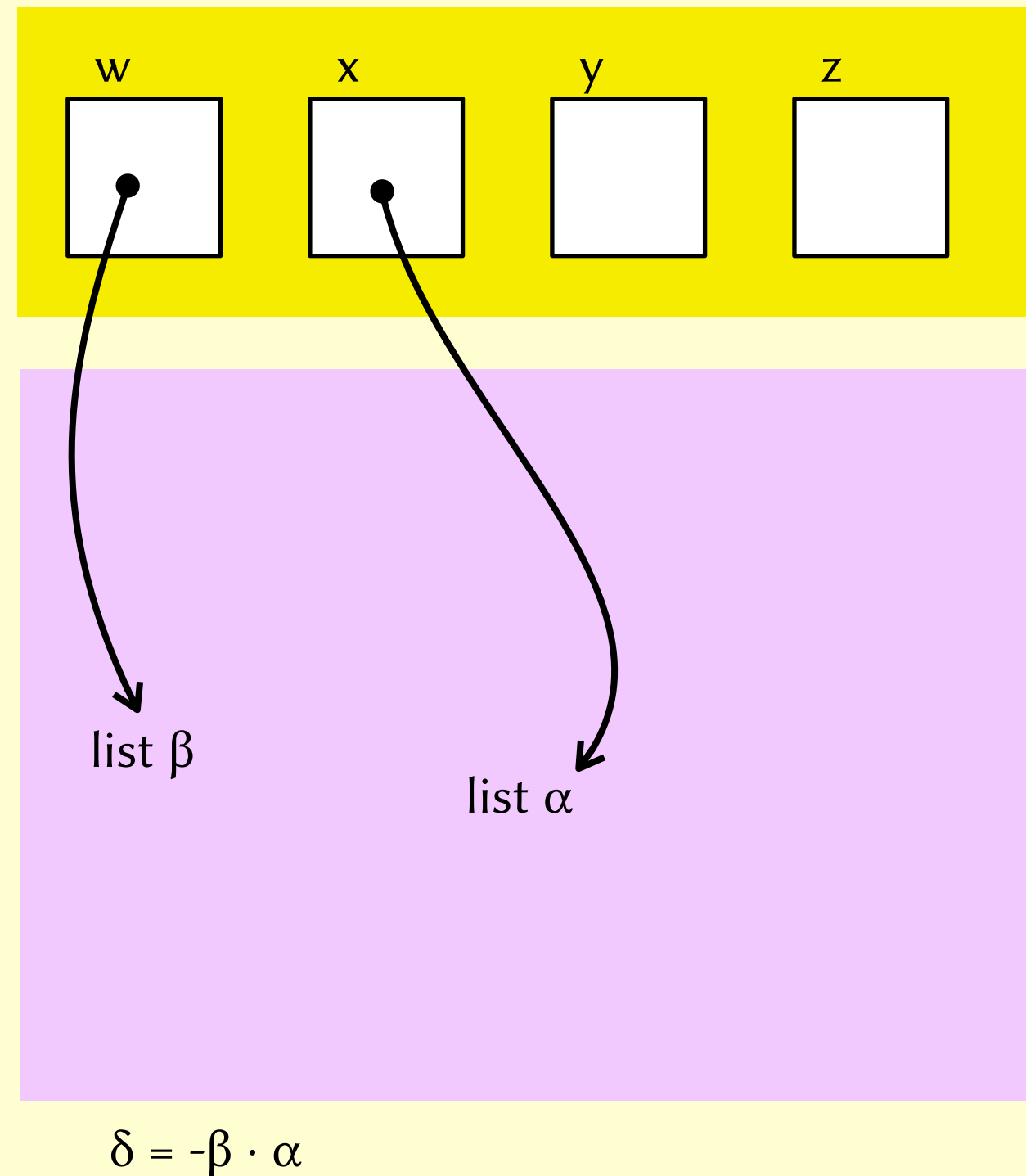
List reverse

```
{list  $\delta$  x}  
w := 0;  
while (x  $\neq$  0) do {  
  z := [x+1];  
  [x+1] := w;  
  w := x;  
  x := z;  
}  
{list  $-\delta$  w}
```



List reverse

```
{list  $\delta$  x}  
w := 0;  
while (x  $\neq$  0) do {  
  z := [x+1];  
  [x+1] := w;  
  w := x;  
  x := z;  
}  
{list  $-\delta$  w}
```

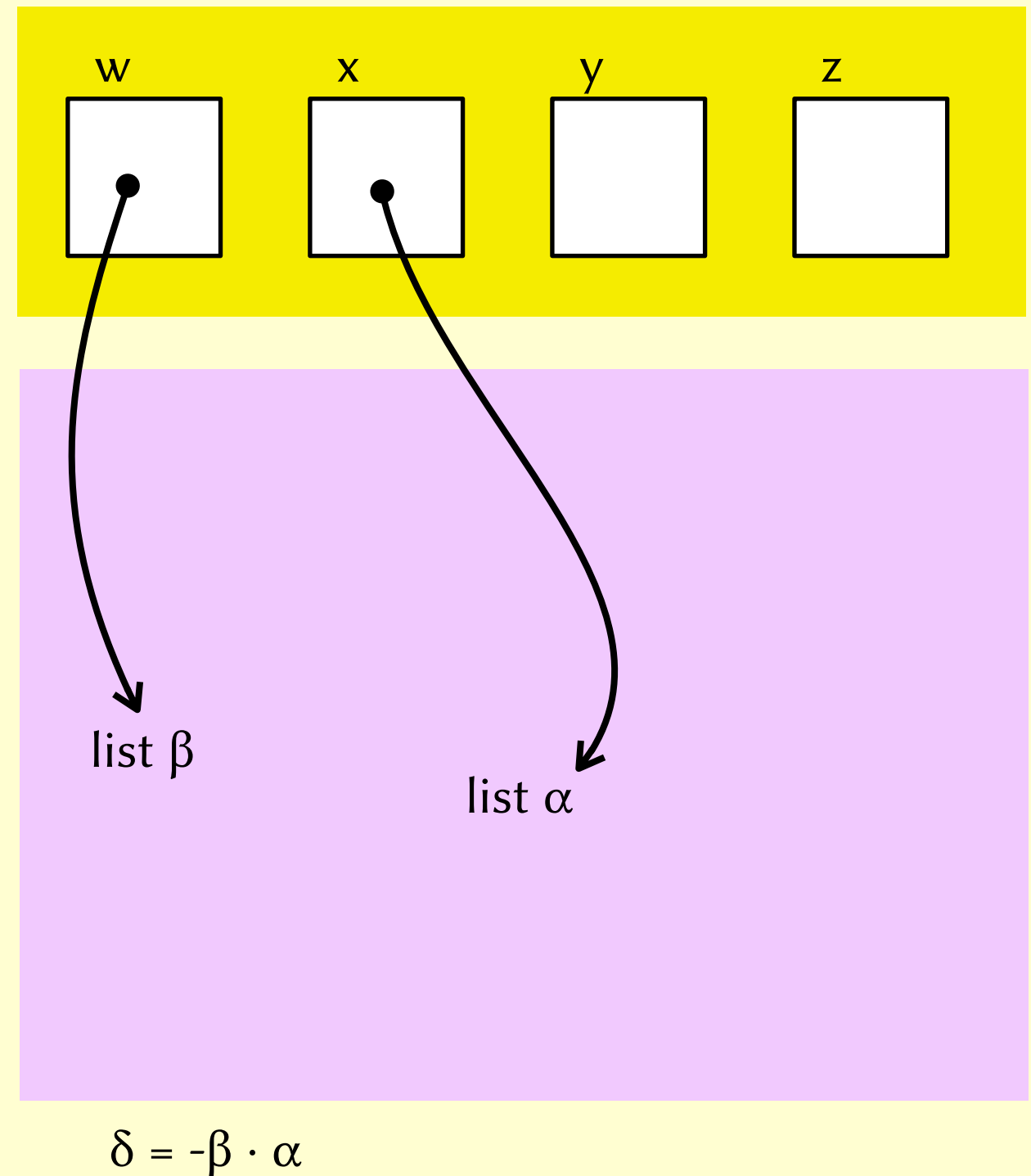


List reverse

```

{list  $\delta$  x}
w := 0;
 $\{\exists \alpha, \beta. \text{list } \alpha \ x \wedge \text{list } \beta \ w \wedge \delta = -\beta \cdot \alpha\}$ 
while (x  $\neq$  0) do {
  z := [x+1];
  [x+1] := w;
  w := x;
  x := z;
}
{list  $-\delta$  w}

```

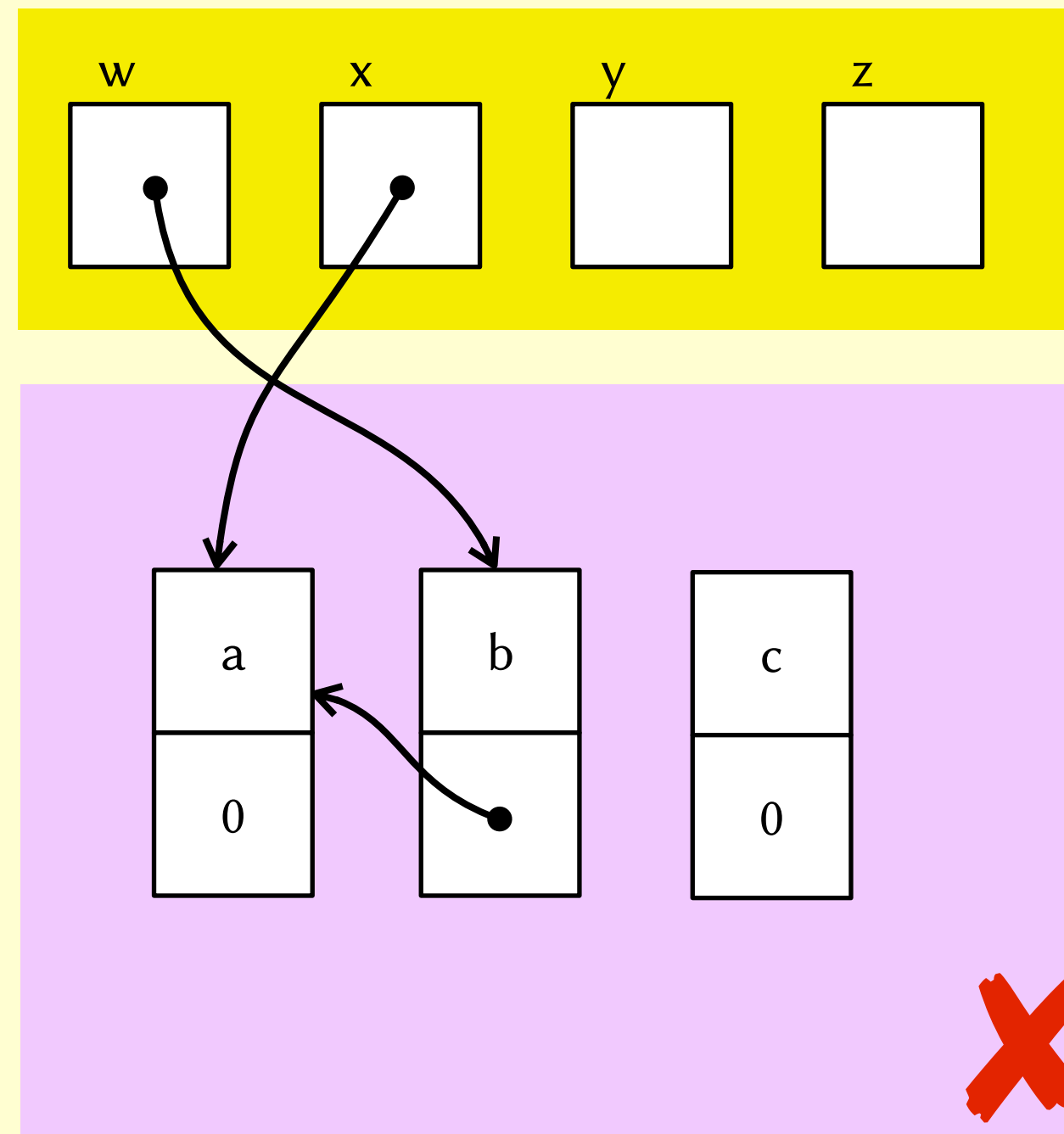


List reverse

```

{list  $\delta$  x}
w := 0;
 $\{\exists \alpha, \beta. \text{list } \alpha \ x \wedge \text{list } \beta \ w \wedge \delta = -\beta \cdot \alpha\}$ 
while (x  $\neq$  0) do {
  z := [x+1];
  [x+1] := w;
  w := x;
  x := z;
}
{list  $-\delta$  w}

```

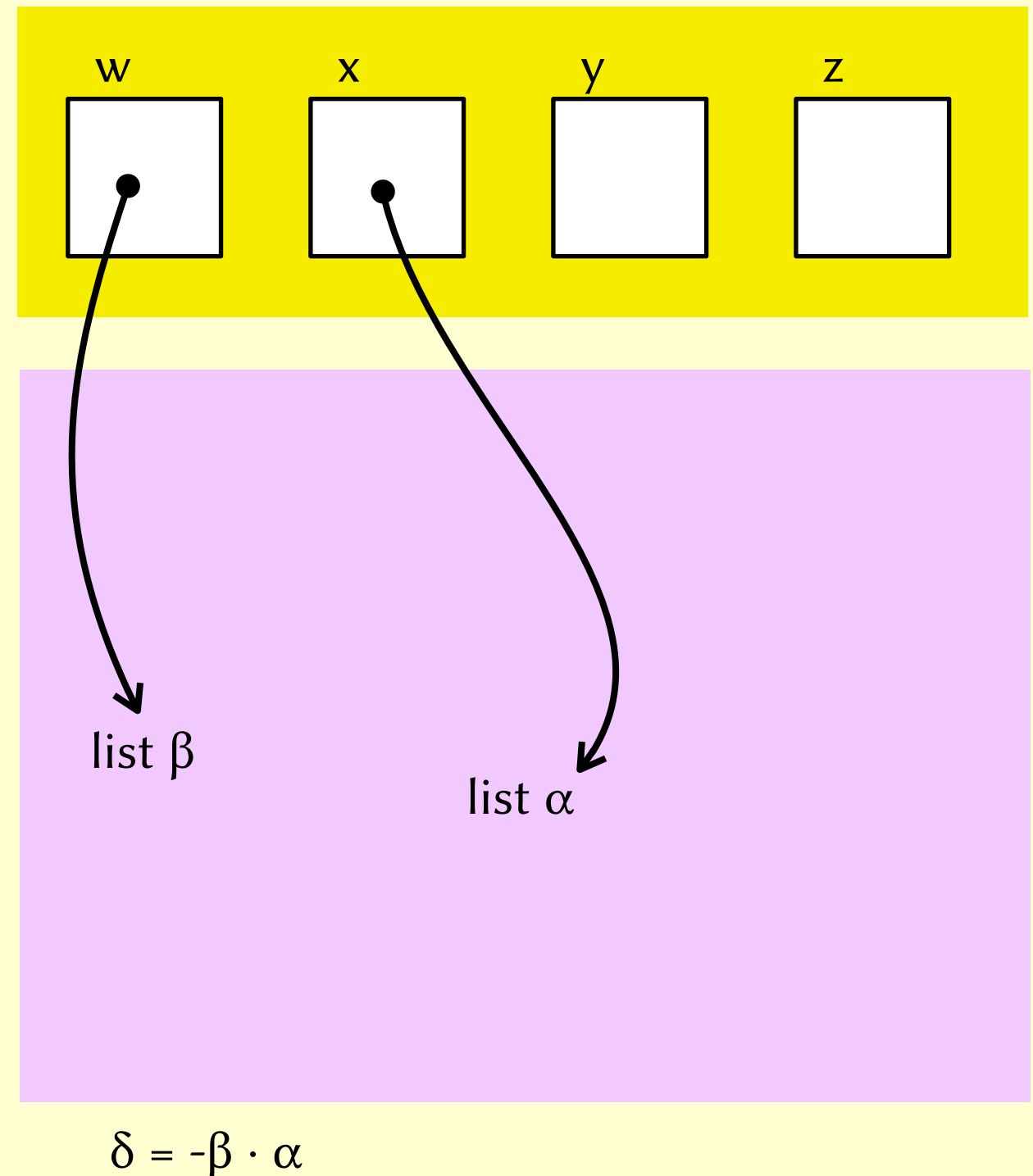


List reverse

```

{list  $\delta$  x}
w := 0;
{ $\exists \alpha, \beta. \text{list } \alpha \ x \wedge \text{list } \beta \ w \wedge \delta = -\beta \cdot \alpha$ 
 $\wedge (\forall z. \text{reach}(x, z) \wedge \text{reach}(w, z) \Rightarrow z=0)$ }
while (x  $\neq$  0) do {
  z := [x+1];
  [x+1] := w;
  w := x;
  x := z;
}
{list  $-\delta$  w}

```



List reverse

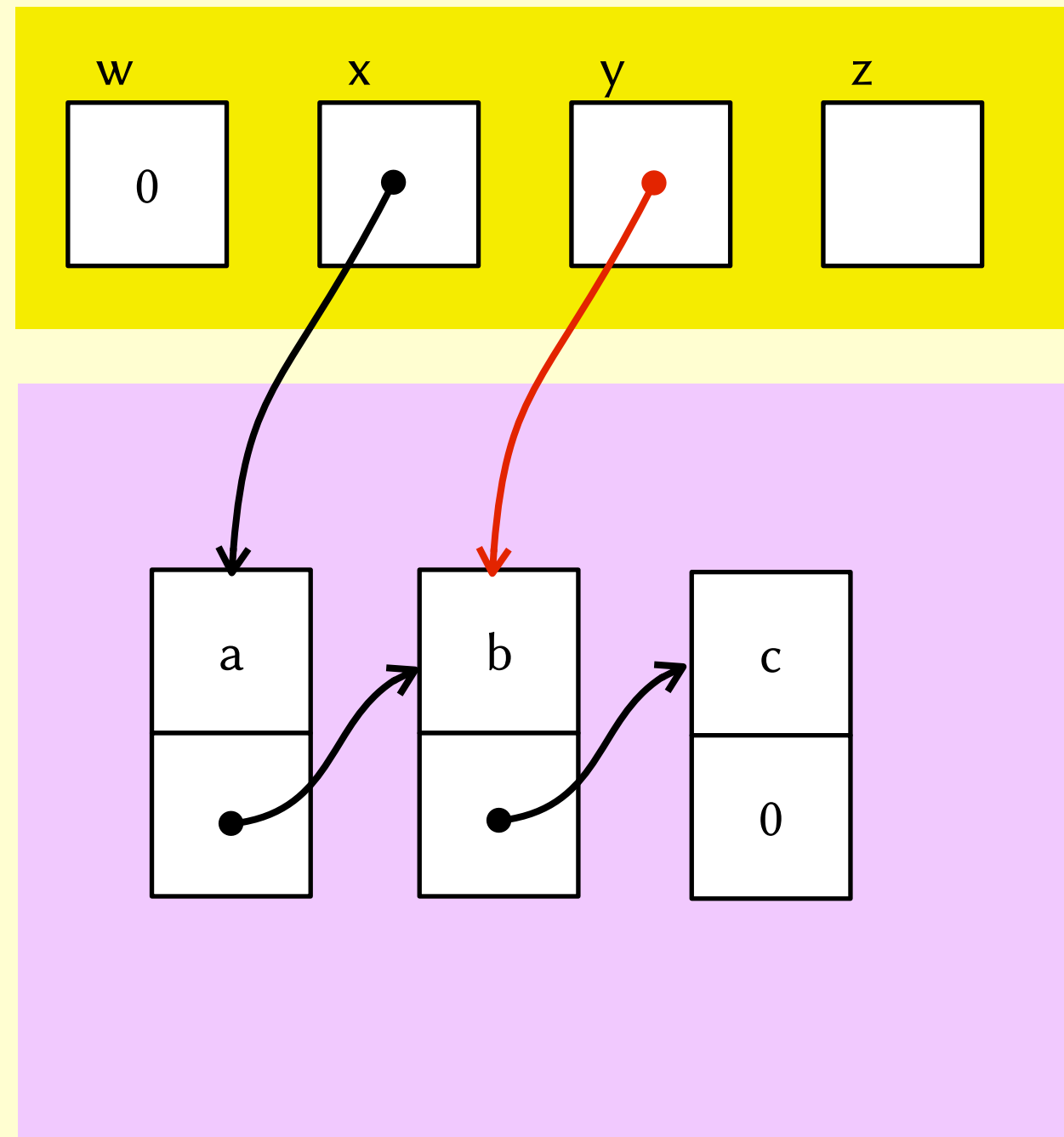
$\{list\ \delta\ x\}$
`list_reverse(x,w)`
 $\{list\ -\delta\ w\}$

List reverse

$\{list\ \delta\ x \wedge list\ \varepsilon\ y\}$
list_reverse(x,w)
 $\{list\ -\delta\ w\}$

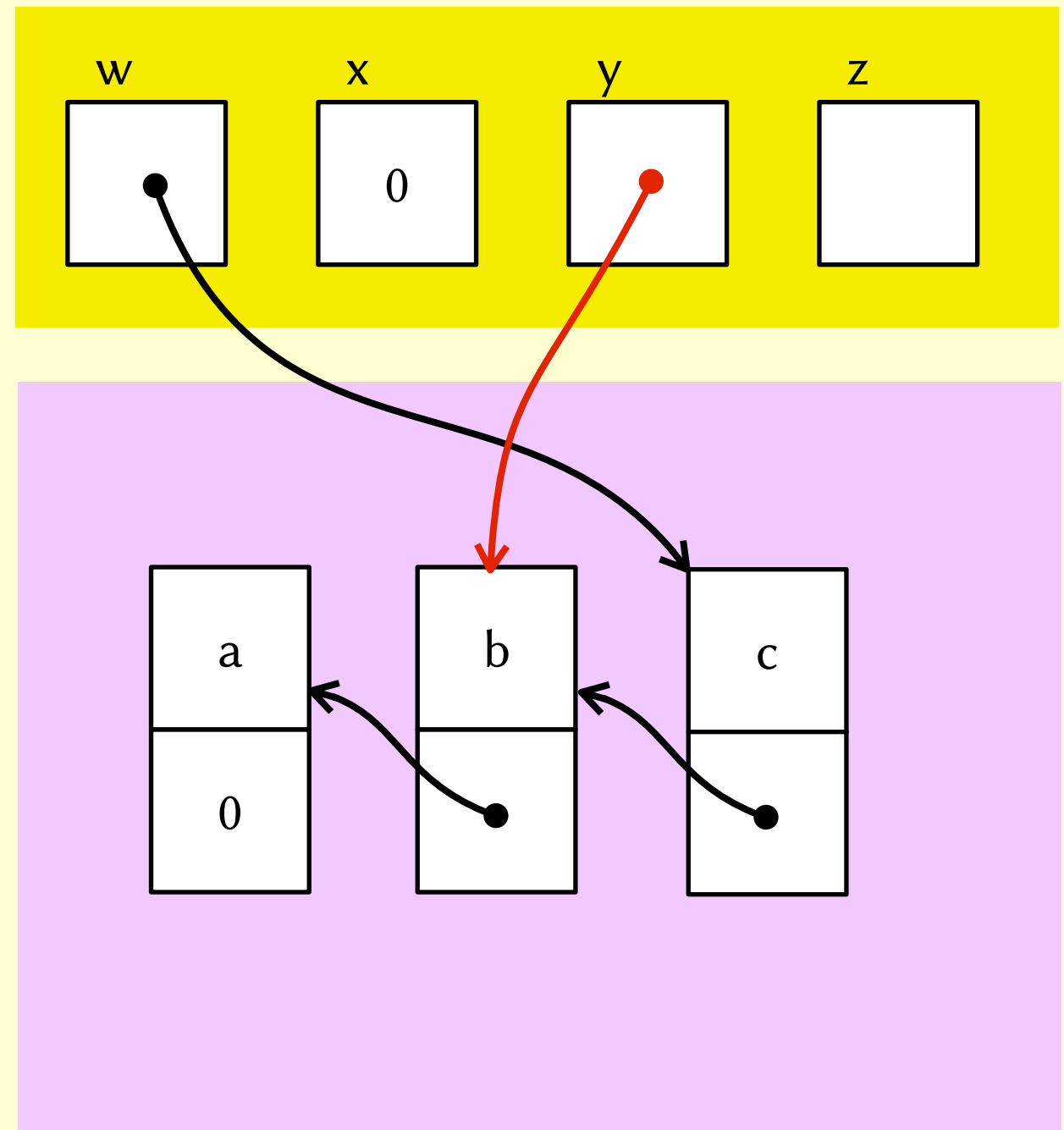
List reverse

$\{list\ \delta\ x \wedge list\ \varepsilon\ y\}$
 $list_reverse(x,w)$
 $\{list\ -\delta\ w\}$



List reverse

$\{list\ \delta\ x \wedge list\ \varepsilon\ y\}$
 $list_reverse(x,w)$
 $\{list\ -\delta\ w\}$



List reverse

$\{ \text{list } \delta \ x \wedge \text{list } \varepsilon \ y$
 $\wedge (\forall z. \text{reach}(x,z) \wedge \text{reach}(y,z) \Rightarrow z=0) \}$
 $\text{list_reverse}(x,w)$
 $\{ \text{list } -\delta \ w \}$

List reverse

```
{list  $\delta$  x  $\wedge$  list  $\varepsilon$  y  
 $\wedge$  ( $\forall z$ . reach(x,z)  $\wedge$  reach(y,z)  $\implies$  z=0)}  
w := 0;  
{ $\exists \alpha, \beta$ . list  $\alpha$  x  $\wedge$  list  $\beta$  w  $\wedge$   $\delta = -\beta \cdot \alpha$   
 $\wedge$  ( $\forall z$ . reach(x,z)  $\wedge$  reach(w,z)  $\implies$  z=0)}  
while (x $\neq$ 0) do {  
  z := [x+1];  
  [x+1] := w;  
  w := x;  
  x := z;  
}  
{list  $-\delta$  w}
```

List reverse

```
{list  $\delta$  x  $\wedge$  list  $\varepsilon$  y  
 $\wedge$  ( $\forall z. \text{reach}(x,z) \wedge \text{reach}(y,z) \implies z=0$ )}  
w := 0;  
{ $\exists \alpha, \beta. \text{list } \alpha x \wedge \text{list } \beta w \wedge \delta = -\beta \cdot \alpha$   
 $\wedge$  ( $\forall z. \text{reach}(x,z) \wedge \text{reach}(w,z) \implies z=0$ )  
 $\wedge$  list  $\varepsilon y$   
 $\wedge$  ( $\forall z. (\text{reach}(x,z) \vee \text{reach}(w,z))$   
   $\wedge \text{reach}(y,z) \implies z=0$ )}  
while (x $\neq$ 0) do {  
  z := [x+1];  
  [x+1] := w;  
  w := x;  
  x := z;  
}  
{list  $-\delta$  w}
```

List reverse

$\{\text{list } \delta \ x \wedge \text{list } \varepsilon \ y$
 $\wedge (\forall z. \text{reach}(x,z) \wedge \text{reach}(y,z) \Rightarrow z=0)\}$
 $w := 0;$
 $\{\exists \alpha, \beta. \text{list } \alpha \ x \wedge \text{list } \beta \ w \wedge \delta = -\beta \cdot \alpha$
 $\wedge (\forall z. \text{reach}(x,z) \wedge \text{reach}(w,z) \Rightarrow z=0)$
 $\wedge \text{list } \varepsilon \ y$
 $\wedge (\forall z. (\text{reach}(x,z) \vee \text{reach}(w,z))$
 $\quad \wedge \text{reach}(y,z) \Rightarrow z=0)\}$
 $\text{while } (x \neq 0) \text{ do } \{$
 $z := [x+1];$
 $[x+1] := w;$
 $w := x;$
 $x := z;$
 $\}$
 $\{\text{list } -\delta \ w \wedge \text{list } \varepsilon \ y$
 $\wedge (\forall z. \text{reach}(w,z) \wedge \text{reach}(y,z) \Rightarrow z=0)\}$

List reverse

- Summary: the proof is fiddly and not modular, but it can be done.

$\{ \text{list } \delta x \wedge \text{list } \varepsilon y$
 $\wedge (\forall z. \text{reach}(x,z) \wedge \text{reach}(y,z) \Rightarrow z=0) \}$
 $\text{list_reverse}(x,w)$
 $\{ \text{list } -\delta w \wedge \text{list } \varepsilon y$
 $\wedge (\forall z. \text{reach}(w,z) \wedge \text{reach}(y,z) \Rightarrow z=0) \}$

Separation Logic



Peter O'Hearn
(1963–)



John C. Reynolds
(1935–2013)

Cigarettes and Whisky

$$(P * Q) s = \exists s_1, s_2. s = s_1 + s_2 \text{ and } (P s_1) \text{ and } (Q s_2)$$

$$(P \wedge Q) s = (P s) \text{ and } (Q s)$$

$$(P \vee Q) s = (P s) \text{ or } (Q s)$$

$$(\neg P) s = \text{not } (P s)$$

$$\text{Cigarettes } s = s \geq \text{£}5$$

$$\text{Whisky } s = s \geq \text{£}20$$

$$\begin{aligned} (\text{Cigarettes} \wedge \text{Whisky}) s &= (\text{Cigarettes } s) \text{ and } (\text{Whisky } s) \\ &= s \geq \text{£}5 \text{ and } s \geq \text{£}20 \\ &= s \geq \text{£}20 \end{aligned}$$

Cigarettes and Whisky

$$(P * Q) s = \exists s_1, s_2. s = s_1 + s_2 \text{ and } (P s_1) \text{ and } (Q s_2)$$

$$(P \wedge Q) s = (P s) \text{ and } (Q s)$$

$$(P \vee Q) s = (P s) \text{ or } (Q s)$$

$$(\neg P) s = \text{not } (P s)$$

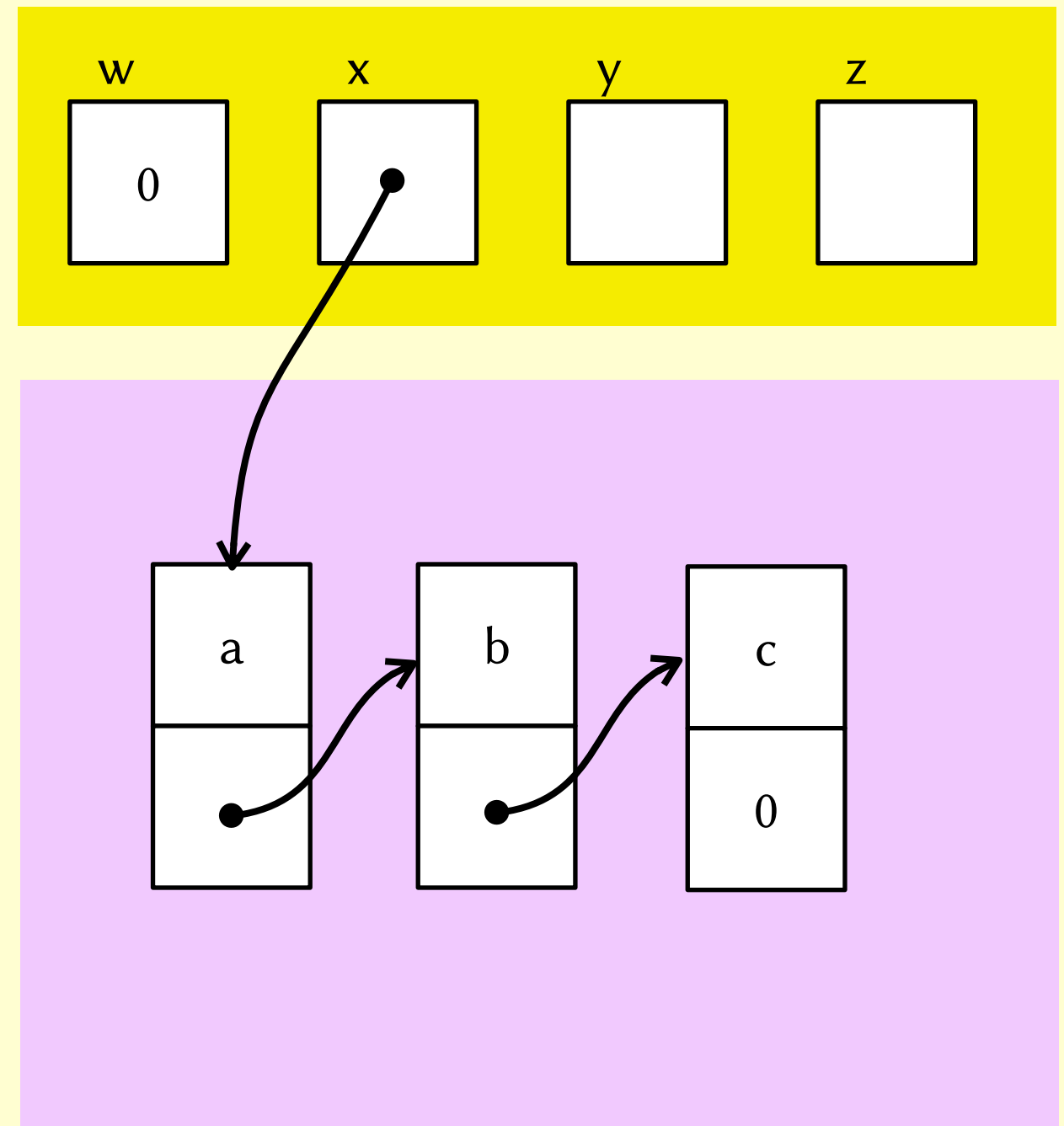
$$\text{Cigarettes} s = s \geq \pounds 5$$

$$\text{Whisky} s = s \geq \pounds 20$$

$$\begin{aligned} (\text{Cigarettes} * \text{Whisky}) s &= \exists s_1, s_2. s = s_1 + s_2 \text{ and } (\text{Cigarettes } s_1) \text{ and } (\text{Whisky } s_2) \\ &= \exists s_1, s_2. s = s_1 + s_2 \text{ and } s_1 \geq \pounds 5 \text{ and } s_2 \geq \pounds 20 \\ &= s \geq \pounds 25 \end{aligned}$$

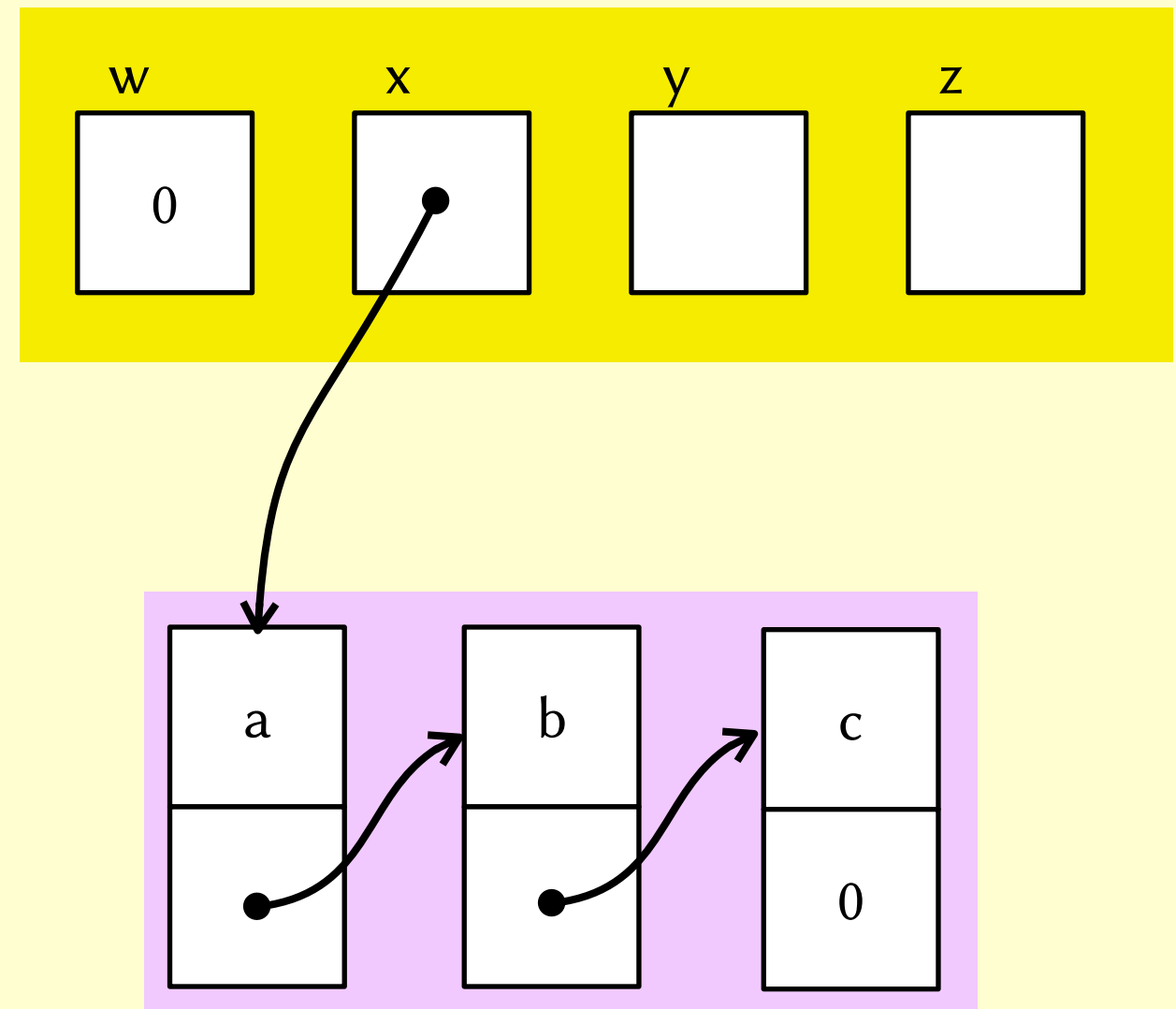
List reverse

```
{list  $\delta$  x}  
w := 0;  
while (x  $\neq$  0) do {  
  z := [x+1];  
  [x+1] := w;  
  w := x;  
  x := z;  
}  
{list  $-\delta$  w}
```



List reverse

```
{list  $\delta$  x}  
w := 0;  
while (x  $\neq$  0) do {  
  z := [x+1];  
  [x+1] := w;  
  w := x;  
  x := z;  
}  
{list  $-\delta$  w}
```

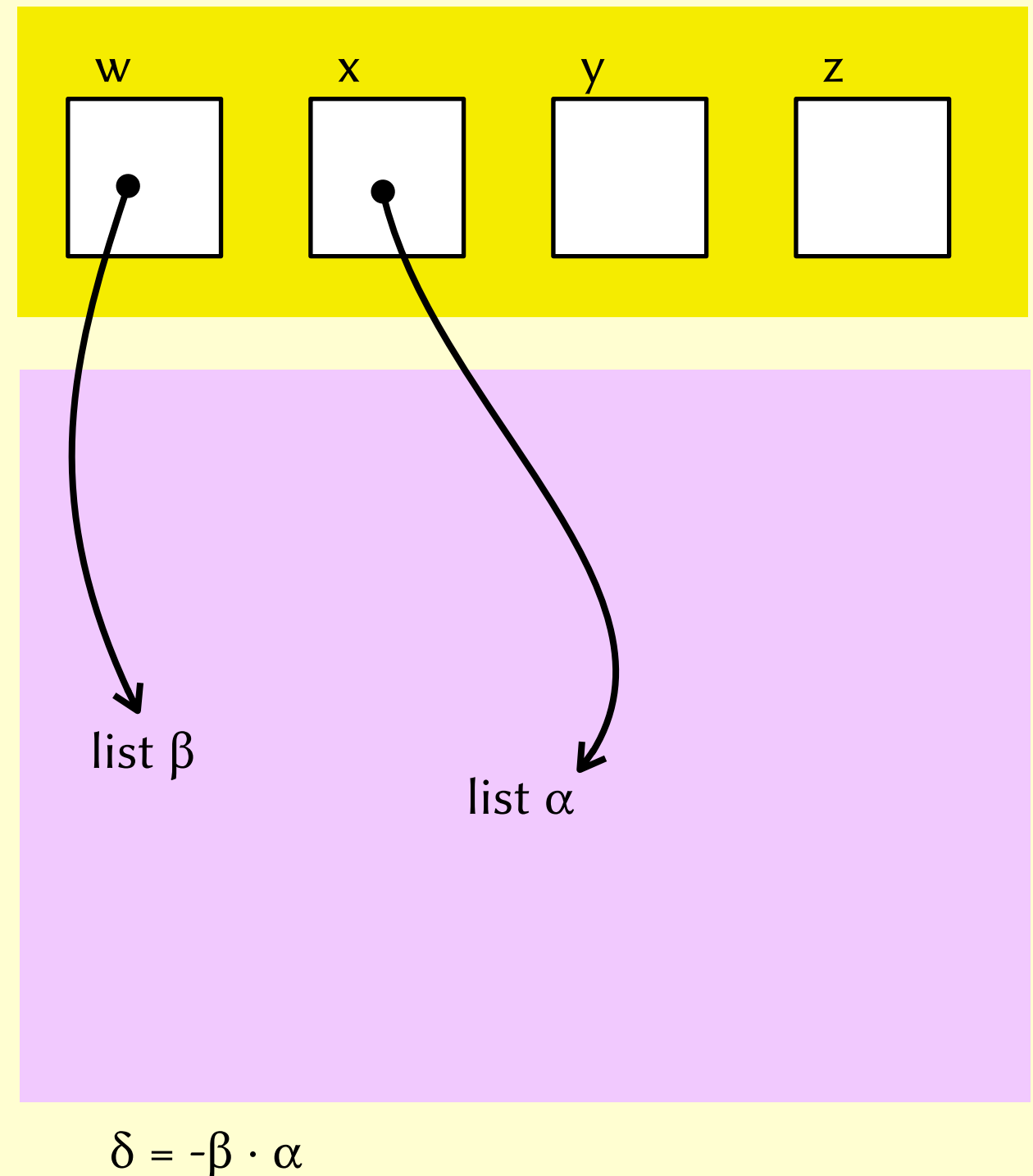


List reverse

```

{list  $\delta$  x}
w := 0;
 $\{\exists \alpha, \beta. \text{list } \alpha \ x \wedge \text{list } \beta \ w \wedge \delta = -\beta \cdot \alpha$ 
 $\wedge (\forall z. \text{reach}(x, z) \wedge \text{reach}(w, z) \Rightarrow z=0)\}$ 
while (x  $\neq$  0) do {
  z := [x+1];
  [x+1] := w;
  w := x;
  x := z;
}
{list  $-\delta$  w}

```



List reverse

$\{\text{list } \delta \ x\}$

$w := 0;$

$\{\exists \alpha, \beta. \text{list } \alpha \ x * \text{list } \beta \ w * \delta = -\beta \cdot \alpha\}$

while $(x \neq 0)$ do {

$z := [x+1];$

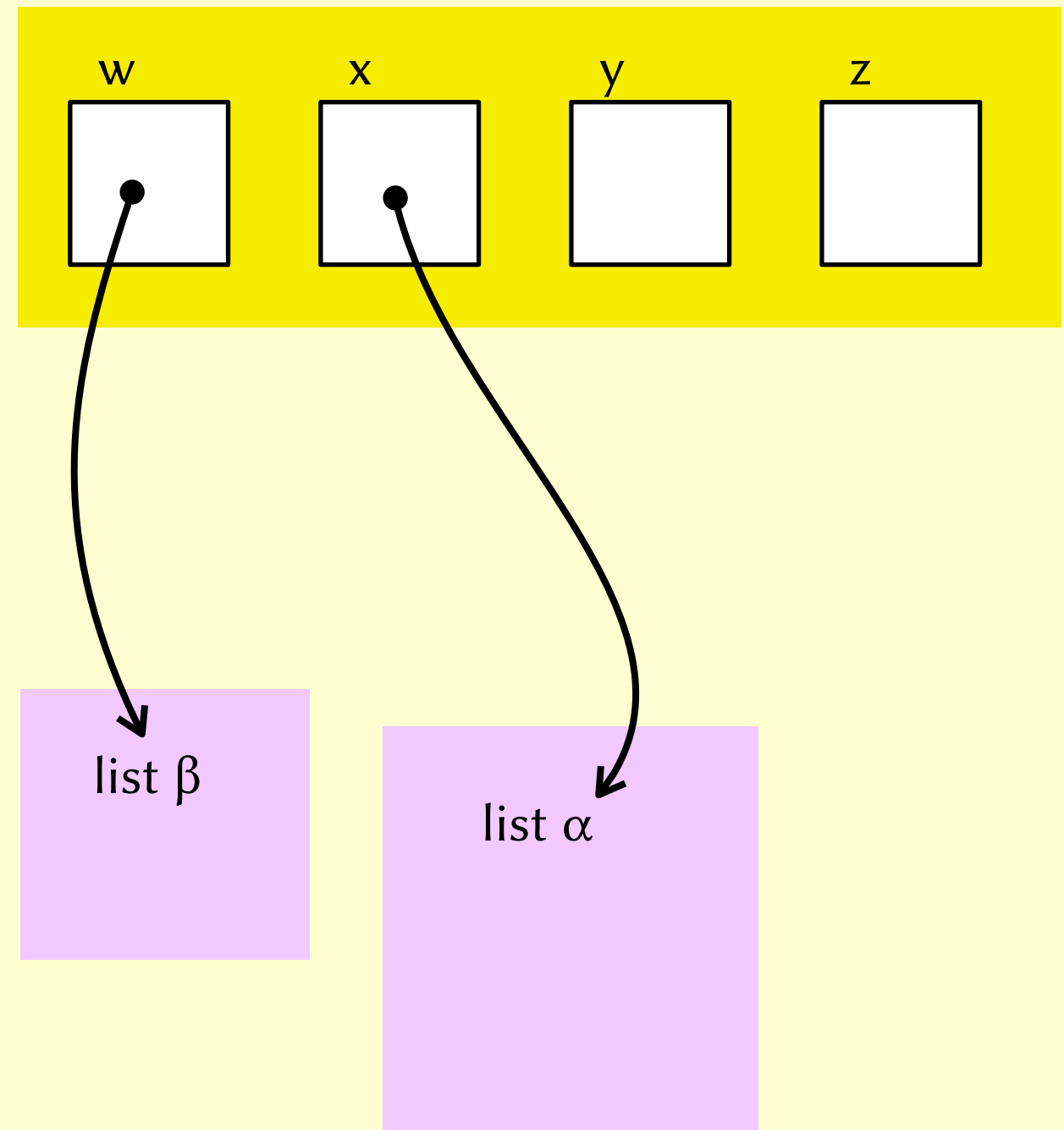
$[x+1] := w;$

$w := x;$

$x := z;$

}

$\{\text{list } -\delta \ w\}$



$$\delta = -\beta \cdot \alpha$$

List reverse

$\{\text{list } \delta \ x\}$

$w := 0;$

$\{\exists \alpha, \beta. \text{list } \alpha \ x * \text{list } \beta \ w * \delta = -\beta \cdot \alpha\}$

while ($x \neq 0$) do {



$z := [x+1];$

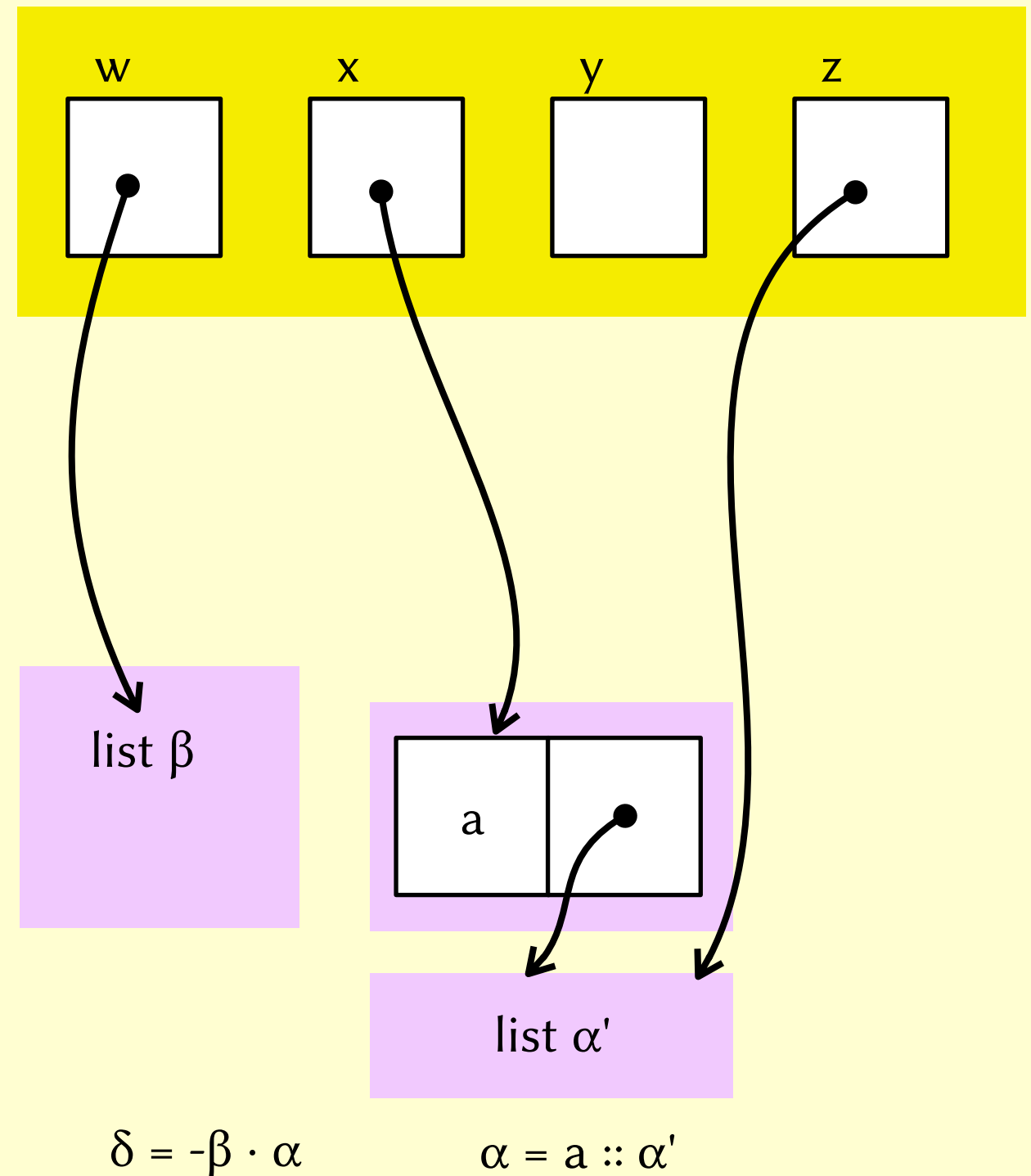
$[x+1] := w;$

$w := x;$

$x := z;$

}

$\{\text{list } -\delta \ w\}$



List reverse

$\{\text{list } \delta \ x\}$

$w := 0;$

$\{\exists \alpha, \beta. \text{list } \alpha \ x * \text{list } \beta \ w * \delta = -\beta \cdot \alpha\}$

while $(x \neq 0)$ do {

$z := [x+1];$

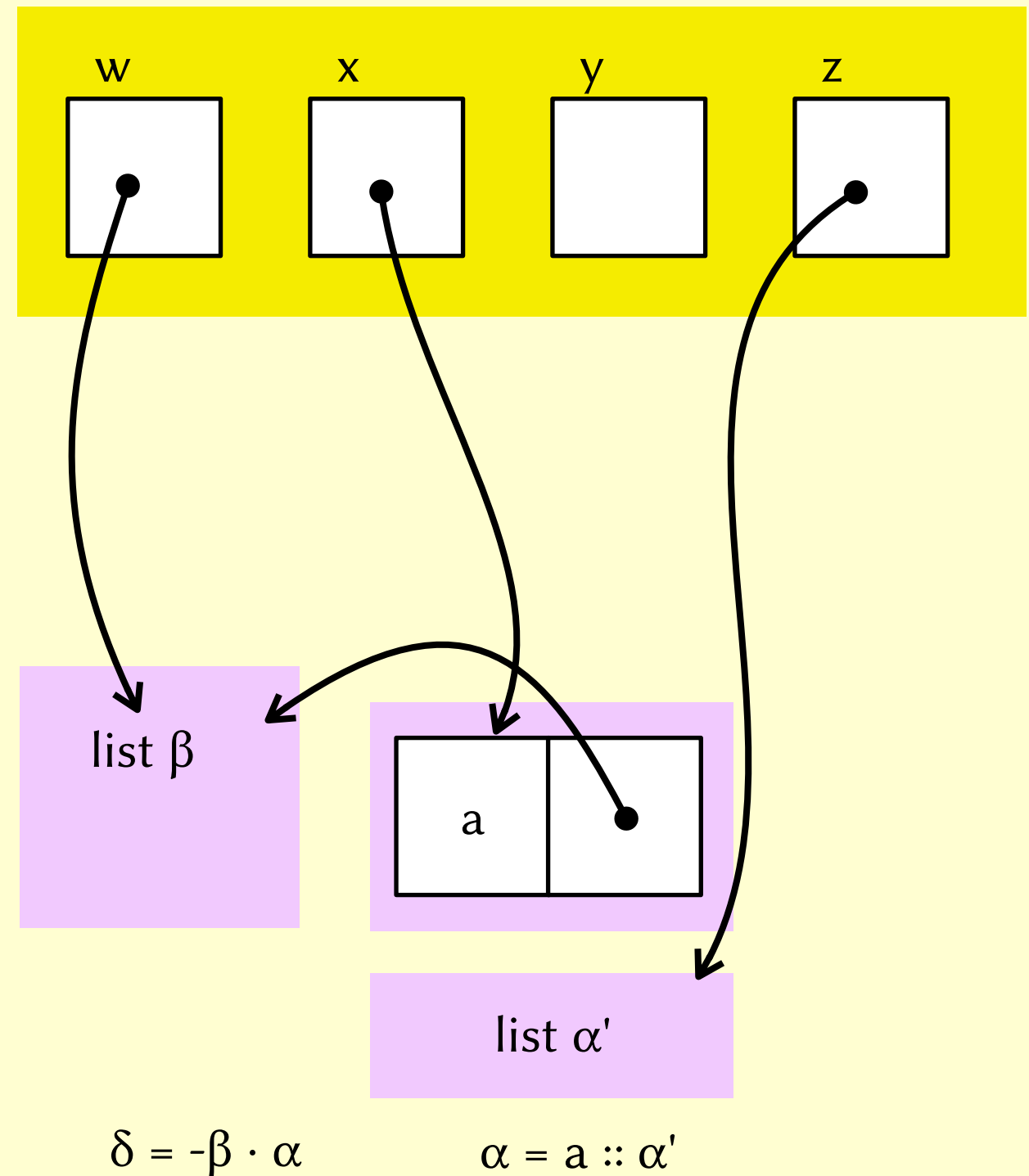
$[x+1] := w;$

$w := x;$

$x := z;$

}

$\{\text{list } -\delta \ w\}$



List reverse

$\{\text{list } \delta \ x\}$

$w := 0;$

$\{\exists \alpha, \beta. \text{list } \alpha \ x * \text{list } \beta \ w * \delta = -\beta \cdot \alpha\}$

while $(x \neq 0)$ do {

$z := [x+1];$

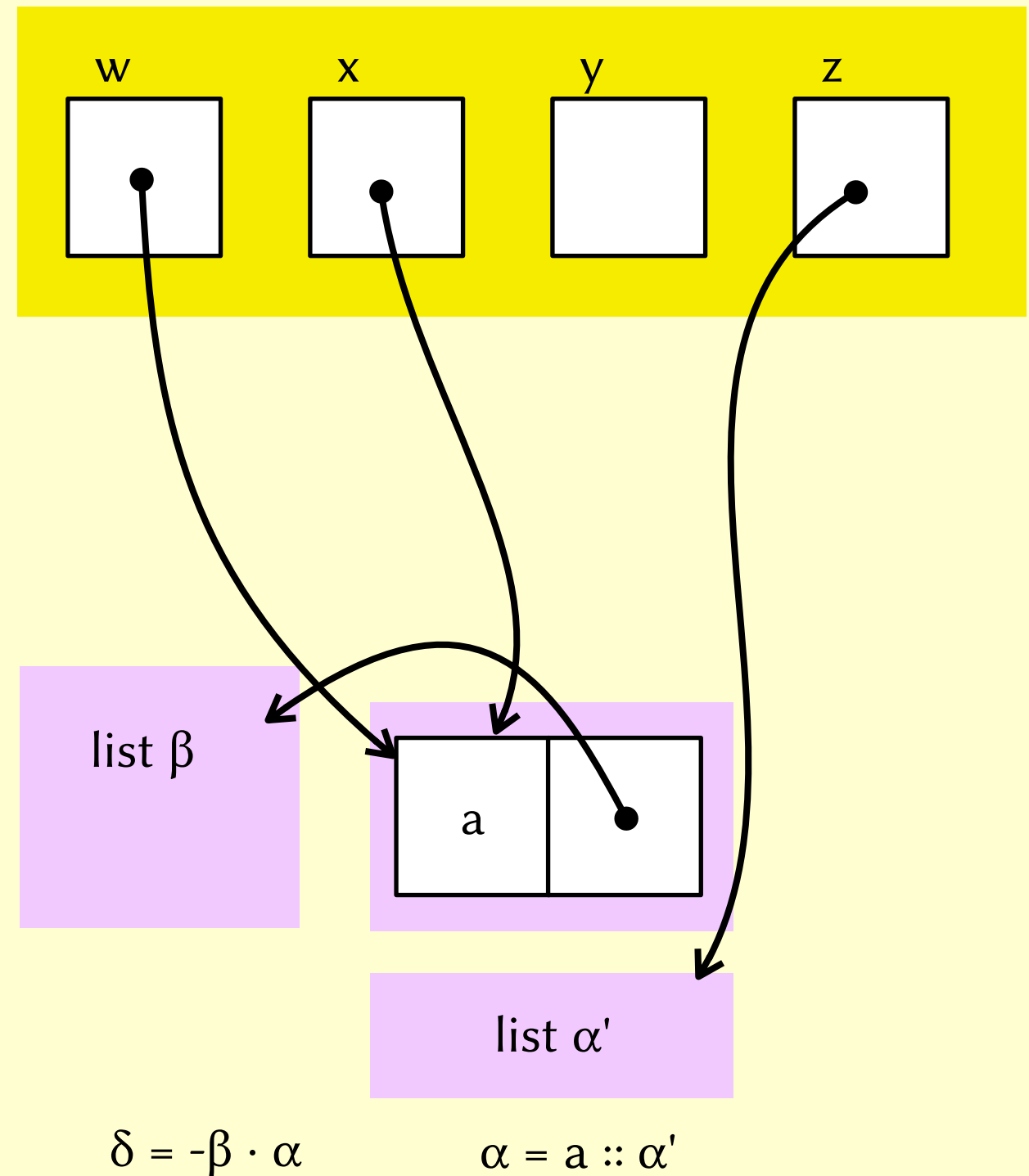
$[x+1] := w;$

$w := x;$

$x := z;$

}

$\{\text{list } -\delta \ w\}$



List reverse

$\{\text{list } \delta \ x\}$

$w := 0;$

$\{\exists \alpha, \beta. \text{list } \alpha \ x * \text{list } \beta \ w * \delta = -\beta \cdot \alpha\}$

while ($x \neq 0$) do {

$z := [x+1];$

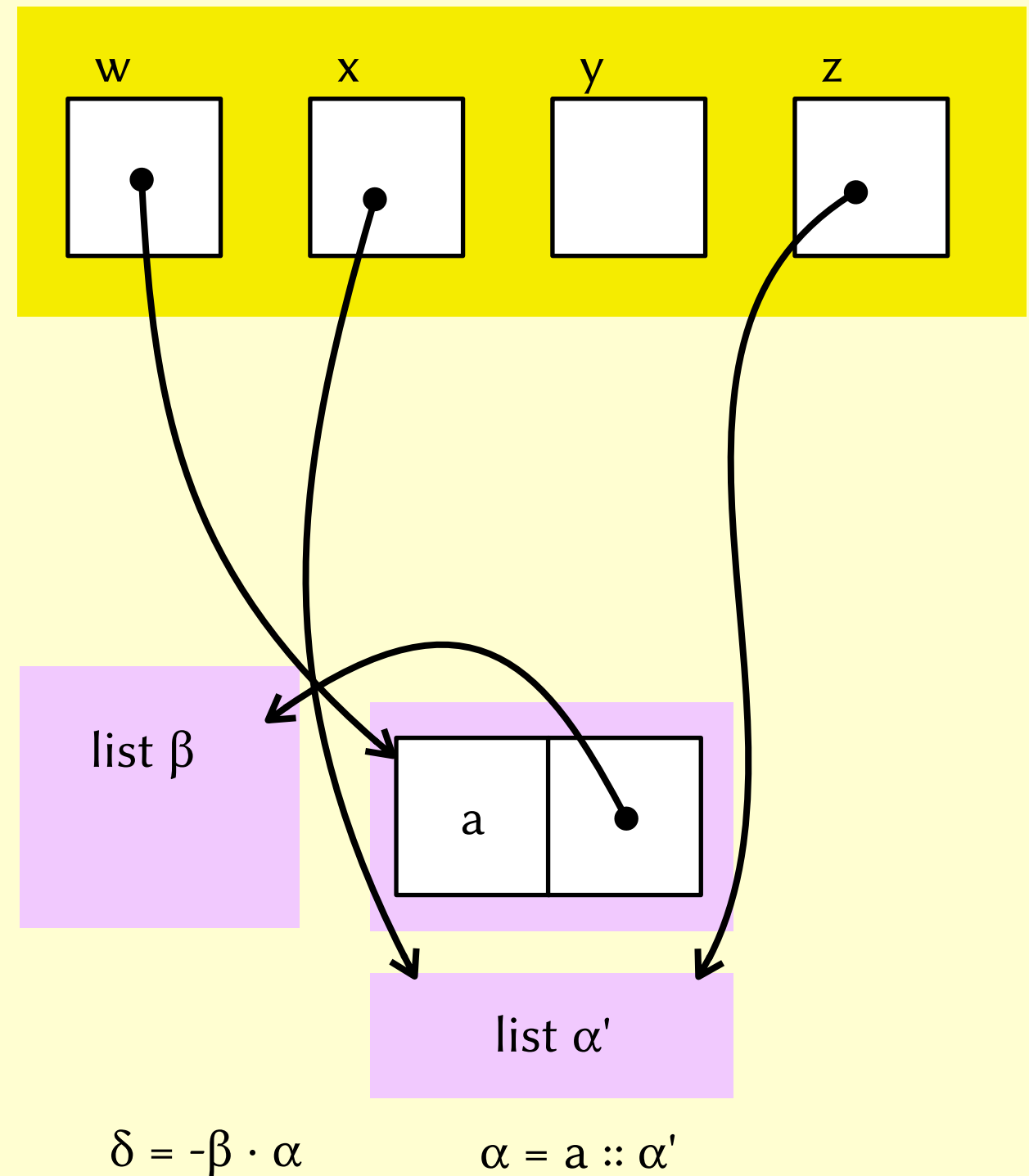
$[x+1] := w;$

$w := x;$

$x := z;$

}

$\{\text{list } -\delta \ w\}$



List reverse

$\{\text{list } \delta \ x\}$

$w := 0;$

$\{\exists \alpha, \beta. \text{list } \alpha \ x * \text{list } \beta \ w * \delta = -\beta \cdot \alpha\}$

while $(x \neq 0)$ do {

$z := [x+1];$

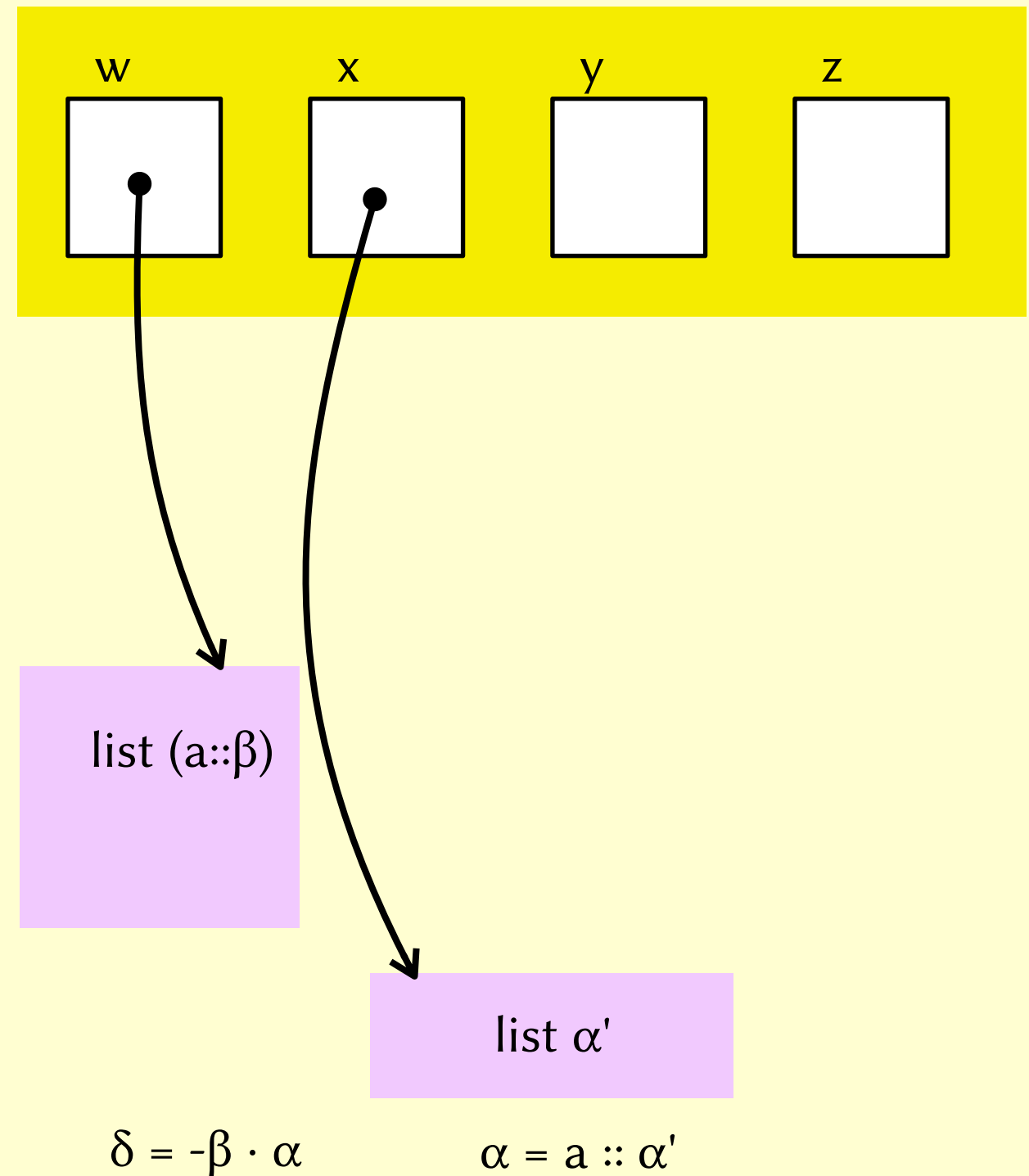
$[x+1] := w;$

$w := x;$

$x := z;$

}

$\{\text{list } -\delta \ w\}$



List reverse

$\{\text{list } \delta \ x\}$

$w := 0;$

$\{\exists \alpha, \beta. \text{list } \alpha \ x * \text{list } \beta \ w * \delta = -\beta \cdot \alpha\}$

while $(x \neq 0)$ do {

$z := [x+1];$

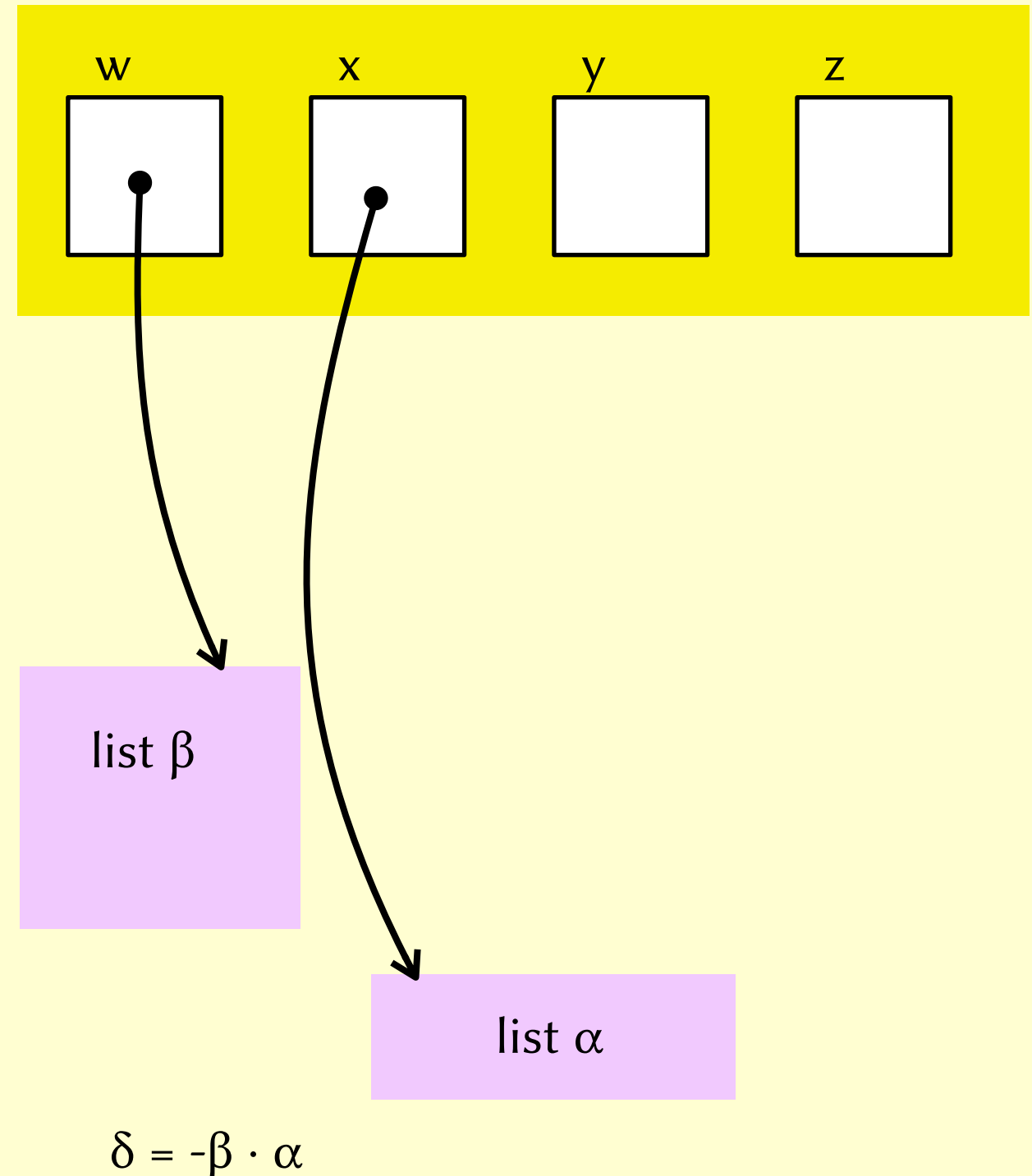
$[x+1] := w;$

$w := x;$

$x := z;$

}

$\{\text{list } -\delta \ w\}$



List reverse

{list δ x}

list_reverse(x,w)

{list $-\delta$ w}

List reverse

$\{ \text{list } \delta x * \text{list } \varepsilon y * \text{tree } t \}$
list_reverse(x,w)
 $\{ \text{list } -\delta w \}$

List reverse

$\{\text{list } \delta x * \text{list } \varepsilon y * \text{tree } t\}$
 $\text{list_reverse}(x, w)$
 $\{\text{list } -\delta w * \text{list } \varepsilon y * \text{tree } t\}$

$$\frac{\{p\} C \{q\} \quad (\dagger)}{\{p * r\} C \{q * r\}}$$

†provided r doesn't mention
any variable modified by C

Conclusion

- Hoare Logic kicked off the field of software verification around 1969.
- Hoare Logic always struggled to reason about heap-allocated data structures.
- Separation Logic provided a solution around 2001.
- This has powered a lot of software verification tools since then, such as Facebook Infer.
- Dafny uses a variant of separation logic called dynamic frames.